8.5

kích thước của biến con trỏ phụ thuộc vào loại kiến trúc của máy tính

Giả sử hệ thống 64-bit có sizeof(double *) == 8 và sizeof(double) == 8. Trên hệ 32-bit, kích thước con trỏ thường là 4 byte.

a) Bộ nhớ được cấp phát ở từng dòng trong main()

double *p[10]; : 10 * 8 = 80 byte

initialize(p + 3); : (1 + 2 + 3 + 4 + 5) * 8 = 120 byte

tổng là 200 byte đã được cấp phát;

b) Cách initialize() cấp phát bộ nhớ

initialize(p + 3) nhận con trỏ tới p[3]. Vòng lặp i = 0..4 thực hiện:

*(p + i) = new double[i + 1];

Do đó, xét theo các phần tử của mảng p trong main():

p[0] -> chưa được khởi tạo

p[1] -> chưa được khởi tạo

p[2] -> chưa được khởi tạo

p[3] -> double[1] (1 phần tử, 8 byte)

p[4] -> double[2] (2 phần tử, 16 byte)

p[5] -> double[3] (3 phần tử, 24 byte)

p[6] -> double[4] (4 phần tử, 32 byte)

p[7] -> double[5] (5 phần tử, 40 byte)

p[8] -> chưa được khởi tạo

p[9] -> chưa được khởi tạo

Tổng cộng có ****5 lần cấp phát**** trên heap, gồm ****15 phần tử double****:

1 + 2 + 3 + 4 + 5 = 15 phần tử

15 * sizeof(double) = 15 * 8 = 120 byte

Năm vùng nhớ này là năm khối riêng biệt; chúng không nhất thiết nằm liền nhau trong bộ nhớ.

c) Viết hàm release()

Các mảng được cấp phát bằng new double[...], nên phải giải phóng bằng delete[]:

void release(double **p)

```
{
   if (p == NULL)
      return;

   for (int i = 0; i < 5; ++i) {
      delete[] *(p + i);
      *(p + i) = NULL;
   }
}
```

Trong main(), phải gọi release(p + 3) vì initialize() đã nhận p + 3:

```
int main()
{
   double *p[10] = {NULL};
   initialize(p + 3);
   release(p + 3);
   return 0;
}
```